

Problema del productor-consumidor
con prioridades y su resolución
con mutexes

Nicolas Corbal Capas
Hugo Veiga Couselo

Introducción

La concurrencia es una característica fundamental en los sistemas operativos modernos y un factor clave para mejorar el rendimiento. Sin embargo, el acceso concurrente a recursos compartidos hace necesario el uso de mecanismos de sincronización. En muchos casos se establece una jerarquía de prioridades de modo que aquellos elementos más prioritarios deben ser procesados preferentemente.

Dentro del clásico problema del productor-consumidor se puede entender como la situación en que varios productores produzcan items de distinta prioridad en una región de datos independiente de los otros productores, y un consumidor deba consumir preferentemente aquellos con prioridad más alta. Así el consumidor no solo debe acceder de forma segura a las regiones críticas, sino que también seleccionar la región crítica que desea acceder.

En el presente informe se documenta la implementación de una solución a este problema utilizando mutexes y variables de condición en lenguaje C, adaptando el código visto en clase para manejar varios buffers y simulando diferentes velocidades de ejecución mediante llamadas a la función `sleep()`. Se analiza el funcionamiento de la solución, las decisiones de diseño adoptadas y las conclusiones obtenidas tras su ejecución.

Forma de ejecución

Primeramente, para compilar el código un ejemplo de comando de compilación usando el compilador gcc es:

```
gcc -Wall -I ./ prod_cons_prioridad.c buffer.c -o ejecutable
```

Siendo el ejecutable ejecutado simplemente con:

```
./ejecutable
```

Funcionamiento del código

La responsabilidad del código vuelve a estar dividida entre 3 archivos, `buffer.c`, `buffer.h` y `prod_cons_prioridad.c`. El archivo `buffer.c` contiene las funciones necesarias para la producción y la consumición, el archivo de cabecera el tipo de elemento que usaremos como forma de acceso común y `prod_cons_prioridad.c` la responsabilidad de la solución del problema.

En tipo de dato `datos` se recoge lo anterior necesario para la cola (un buffer y marcadores de inicio y fin) adicionalmente se añaden las variables de condición y el mutex como variante a usar variables globales. Además, se añade el entero `prioridad` para que cada productor sepa que prioridad tiene y en que buffer debe escribir.

El archivo `prod_cons_prioridad.c` se divide en funciones. La función `main` que se encarga de inicializar las variables y lanzar los hilos, así como de esperar a su finalización para la liberación de recursos y los hilos productores y consumidor.

El funcionamiento del hilo productor es similar a un productor normal, espera un tiempo para simular un tiempo de producción, produce un ítem y adquiere el mutex. Si el mutex estuviese bloqueado el hilo se bloquea y espera a que el consumidor libere el mutex, luego realiza lo análogo con la variable de condición. Si hay espacio en el buffer inserta el ítem, sino la variable de condición se emplea para usar `pthread_cond_wait()` y llamar al consumidor. Luego avisa a la variable de condición con `pthread_cond_signal` y libera el mutex.

```
for(int i = 0; i < MAX_ITER; i++){
    sleep(1 + rand()%6); // agarda aleatoria 1-6 s
    item = produce_item(arq, &suma);
    if(item < 1 || item > 99) break;

    pthread_mutex_lock(&args->mutex);
    while(args->cantidad >= N) pthread_cond_wait(&args->condp, &args->mutex);
    insert_item(item, args);
    printf("[%ld] P%d insire %d\n", tempo_ms(), args->prioridade, item);
    pthread_cond_signal(&args->condc);
    pthread_mutex_unlock(&args->mutex);
}
```

El consumidor sin embargo cambia su comportamiento para respetar la prioridad. Entra en un bucle indefinido hasta que los tres productores terminaron. Entonces ejecuta un bucle donde

primero siempre se comprueba la situación del primer productor. Si no terminó ejecuta `pthread_mutex_trylock()` que no bloquea el hilo si no puede adquirir el mutex. Si el retorno no es 0 eso significa que no adquirió el mutex y no puede entrar en la región crítica por lo que debe comprobar en el buffer de prioridad inmediatamente inferior que se hace con el con la cláusula `continue` expuesta. Al entrar comprueba la cantidad de elementos, si hay 0 debe también pasar al siguiente buffer pero liberando primero al mutex. En el caso de que haya elementos que consumir, consume un elemento y avisa al productor correspondiente por si está bloqueado esperando a que el consumidor consuma. Luego se añade a la suma y se simula un procesado. La variable `feito` sirve para reiniciar cada vez que se consume, si se consume un elemento de prioridad 1 puede que aun haya otro por lo que se debe comprobar de nuevo el buffer primero; un elemento de prioridad n se consume si en las $n-1$ iteraciones anteriores no se consumió nada.

El hecho de que un proceso tenga mayor prioridad no implica necesariamente que termine primero pues sus inserciones dependen de la planificación que marca el scheduler, no el código. Sin embargo, los procesos de mayor prioridad tienen mayor probabilidad de terminar antes.

```
while(!(fin[0] && fin[1] && fin[2])){
    int feito = 0;
    // probar buffers por orde de prioridade con trylock
    for(int p = 0; p < NP && !feito; p++){
        if(fin[p]) continue;
        if(pthread_mutex_trylock(&buffers[p].mutex) != 0) continue;
        if(buffers[p].cantidad > 0){
            item = remove_item(&buffers[p]);
            pthread_cond_signal(&buffers[p].condp);
            pthread_mutex_unlock(&buffers[p].mutex);
            if(item == 0){
                fin[p] = 1;
                printf("[%ld] Cons fin de P%d\n", tempo_ms(), p+1);
            } else {
                printf("[%ld] Cons consume %d de P%d\n", tempo_ms(), item, p+1);
                sumas[p] += item;
                sleep(1 + rand()%3); // procesado 1-3 s
            }
            feito = 1;
        } else {
            pthread_mutex_unlock(&buffers[p].mutex);
        }
    }
    if(!feito) nanosleep(&ts, NULL);
}
```

Conclusión

La implementación desarrollada demuestra que es posible resolver el problema del productor-consumidor con prioridades utilizando únicamente mutexes y variables de condición, manteniendo la seguridad en el acceso concurrente y respetando la jerarquía de prioridades establecida entre los productores. El uso de buffers independientes, cada uno asociado a un mutex, permite que los productores trabajen en paralelo sin condiciones de carrera, mientras que el consumidor puede seleccionar qué región crítica atender en cada momento.

El empleo de `pthread_mutex_trylock()` en el consumidor resulta clave para evitar bloqueos innecesarios y garantizar que los ítems de mayor prioridad sean atendidos antes que los de prioridad inferior. Así al combinar con las variables de condición se asegura que el hilo consumidor no se bloquee. Se considera que no hay espera activa dado que si después de comprobar en los tres buffers no hay consumición se cede la cpu a otro hilo.

Los resultados obtenidos muestran que la prioridad influye directamente en el orden de consumo, aunque no determina estrictamente el orden de finalización de los hilos, ya que este depende también de la planificación del sistema operativo y de los tiempos de producción simulados. Aun así, el comportamiento observado confirma que la solución respeta la política de prioridades y evita condiciones de carrera, interbloqueos y otros problemas típicos de la concurrencia.

En conjunto, la solución implementada es correcta, robusta y extensible, y constituye un ejemplo práctico de cómo combinar mecanismos de sincronización básicos para resolver un problema concurrente real con requisitos adicionales como la gestión de prioridades.